

Laboratoire 2

Application JavaFX — Version 1

Consignes générales à lire AVANT de choisir un sujet

Cours	Remise	Notation
420-930-MA (Été 2026)	Vendredi 11 septembre 2026, 23h59	Voir barème (page 6)
Équipe	Format de remise	Où
2 à 3 personnes	Fichier README.md + lien GitHub public	Teams — canal du groupe

IMPORTANT — À lire absolument :

Ce laboratoire est la **première moitié du projet final** de la session. Le **Laboratoire 3** viendra étendre CE MÊME projet (ajout d'une base de données PostgreSQL, authentification, multithreading, statistiques). **La qualité architecturale de votre Lab 2 déterminera la difficulté de votre Lab 3.**

Un code sale au Lab 2 = un cauchemar au Lab 3. Un code propre au Lab 2 = une extension facile au Lab 3.

Traitez ce code comme du code de production. Pas comme un devoir.

1. Objectifs pédagogiques

À la fin du Laboratoire 2, votre équipe aura :

- Conçu et implémenté une **architecture MVC propre** en Java
- Développé une **application JavaFX** fonctionnelle avec FXML + CSS
- Codé **à la main** au moins 3 algorithmes de tri (au choix parmi Bubble, Selection, Insertion, Merge, Quick)
- Implémenté un système de **pagination, filtrage multi-critères** et **recherche**
- Utilisé **Git et GitHub** de façon professionnelle (commits, branches, pull requests)
- Rédigé une **documentation claire** (README, javadoc minimal)

2. Choix du sujet

Vous choisissez UN sujet parmi les 5 proposés (fichiers séparés) :

No	Sujet	Thème
1	Netflix Catalog	Films/séries
2	Voitures d'occasion	Automobile
3	Spotify Playlist Manager	Musique
4	NHL Stats Manager	Sport
5	Immobilier (Centris/Realtor)	Habitation

Chaque sujet a son propre document PDF avec :

- Le contexte métier
- Le vocabulaire (glossaire)
- La liste des concepts/données à modéliser
- Les fonctionnalités spécifiques au sujet

— Les bonus au choix

Vous devez lire ce document ET le PDF du sujet choisi avant de commencer à coder.

3. Contraintes techniques COMMUNES à tous les sujets

3.1 Stack technique imposée

- **Java 17 ou supérieur** (recommandé : Java 21 LTS)
- **JavaFX 21** pour l'interface graphique
- **Maven** comme gestionnaire de dépendances (fichier `pom.xml`)
- **FXML** pour définir les vues (fait au moins avec Scene Builder)
- **CSS** pour le style
- **Git** pour la gestion de version, hébergé sur **GitHub**

Interdit : Swing, AWT, autres frameworks UI, outils IA sans mention explicite.

3.2 Données

Les données de votre application proviennent d'un fichier **CSV ou JSON** statique que vous chargez au démarrage. Chaque sujet précise le minimum de **lignes de données attendues** (généralement 200 à 500).

Pourquoi CSV/JSON et pas de base de données ? Parce que la base de données est le sujet du **Laboratoire 3**. Ne mettez PAS de PostgreSQL au Lab 2 — vous vous simplifierez la vie au Lab 3.

4. Contraintes ARCHITECTURALES (INSISTANCE MAXIMALE)

Cette section détermine 40% de la note. Lisez-la TRÈS attentivement.
L'objectif : produire du code que vous pourrez **ÉTENDRE** au Lab 3 sans le réécrire.

4.1 Structure de packages OBLIGATOIRE

Votre projet DOIT respecter exactement cette organisation :

Package	Contenu autorisé
[projet]	Uniquement <code>MainFx.java</code> (point d'entrée)
[projet].model	Classes du domaine métier (POJOs, enums)
[projet].service	Logique métier (filtrage, pagination, recherche)
[projet].algorithmes	Interfaces + implémentations des tris
[projet].controller	Contrôleurs JavaFX (liés aux FXML)
[projet].util	Utilitaires (LecteurCSV, formateurs, etc.)

4.2 Règles de séparation strictes

Règle 1 — Le modèle ne connaît PAS l'interface :

- Aucune classe de `model` n'importe quoi que ce soit de `javafx.*`
- Aucun `System.out.println` dans `model` ou `service`
- Le modèle doit être testable sans lancer JavaFX

Règle 2 — La vue (FXML) ne contient PAS de logique métier :

- Aucun calcul dans les fichiers `.fxml`
- Les contrôleurs délèguent le travail aux services

Règle 3 — Les contrôleurs sont MINCES :

- Un contrôleur reçoit un événement, appelle un service, met à jour la vue
- Un contrôleur ne trie PAS lui-même, ne filtre PAS lui-même, ne cherche PAS lui-même
- Toute logique complexe est déportée dans un **service** ou un **algorithme**

Règle 4 — Programmez CONTRE des INTERFACES :

- Vos tris implémentent une interface **Algorithme** (ou nom équivalent)
- Vos services de chargement de données implémentent une interface (**SourceDonnees** par exemple), même si l'unique implémentation est **LecteurCSV** — au Lab 3 vous ajouterez **LecteurBD**

4.3 Nomenclature obligatoire

- Noms de classes en **PascalCase**, sans abréviations obscures
- Noms de variables et méthodes en **camelCase**, en français ou anglais mais **cohérent** dans tout le projet
- Noms de packages en minuscules, sans underscore
- Constantes en **MAJUSCULES_SNAKE_CASE**
- Aucun fichier de plus de 300 lignes (si un fichier explose, il faut le découper)

4.4 Ce qui est INTERDIT

- ✗ Utiliser `Collections.sort()` ou `Arrays.sort()` ou `Stream.sorted()` pour VOS boutons de tri
— vous devez appeler VOS algorithmes
- ✗ Mettre toute la logique dans un seul fichier
- ✗ Utiliser des variables globales publiques (`public static ... mutables`)
- ✗ Copier-coller du code entre 2 endroits (extraire une méthode/classe)
- ✗ Laisser des `System.out` de debug dans le code final
- ✗ Laisser du code commenté "au cas où" (utiliser Git pour ça)
- ✗ Utiliser des librairies de tri externes (Guava, Apache Commons)

5. Fonctionnalités OBLIGATOIRES (communes)

5.1 Écran principal

- Une liste ou tableau qui affiche les données du sujet
- Une barre de filtres/recherche (voir sections ci-dessous)
- Une pagination fonctionnelle (voir section)
- Un panneau détail qui affiche l'élément sélectionné

5.2 Tris

- Interface **Algorithme** avec au moins les méthodes : `nom()`, `complexiteTheorique()`, `trier(...)`
- Au moins **3 implémentations** parmi : Bubble, Selection, Insertion, Merge, Quick

- Chaque tri accepte un **Comparator** pour trier selon différents critères
- Boutons ou menu pour trier selon au moins **3 critères différents** (spécifiques au sujet)
- Chaque tri utilise VOS implémentations (interdit `Collections.sort`)

5.3 Pagination

Le tableau affiche au maximum N éléments par page (valeur configurable, par défaut 25) avec :

- Boutons "Précédent" / "Suivant"
- Affichage du numéro de page (ex : "Page 3 / 20")
- Optionnel : ComboBox pour changer la taille de page (10 / 25 / 50 / 100)
- La pagination fonctionne SUR les résultats filtrés (pas sur toutes les données brutes)

5.4 Filtrage multi-critères

Le sujet précise les critères, mais vous devez implémenter au moins **3 filtres qui se combinent** (les 3 s'appliquent simultanément). Chaque filtre est représenté par un contrôle JavaFX approprié (ComboBox, Slider, TextField, DatePicker...).

5.5 Recherche par texte

Un **TextField** qui filtre en temps réel (au fur et à mesure de la frappe). La recherche est :

- Insensible à la casse
- Insensible aux accents (bonus)
- Appliquée sur au moins 2 attributs (spécifiés dans le sujet)

5.6 Compareur de tris (benchmark)

Un écran ou une fenêtre secondaire qui :

- Lance vos 3 (ou 5) tris sur les données complètes
- Mesure le temps d'exécution (`System.nanoTime()`)
- Affiche les résultats dans un tableau ou un graphique

6. Utilisation de Git et GitHub

Le dépôt GitHub est **PUBLIC** et **OBLIGATOIRE**. Sans dépôt public accessible, le laboratoire n'est pas noté.

6.1 Configuration du dépôt

1. Créer un dépôt sur GitHub avec un nom clair (ex : `netflix-catalog-lab2`)
2. Visibilité **Public**
3. Ajouter un `.gitignore` adapté à Java + Maven + IntelliJ
4. Ajouter tous les membres de l'équipe comme collaborateurs

6.2 Commits

- **Minimum 15 commits** au total sur toute la période (indicateur d'un vrai développement itératif)

- Chaque membre de l'équipe DOIT avoir des commits sous son nom
- Messages de commit clairs et en français ou anglais (cohérence)
- Éviter les commits type "update", "fix", "wip" tout seuls
- Style recommandé : "feat : ajout du tri par prix", "fix : pagination incorrecte", "refactor : extraction service de filtrage"

6.3 Branches (recommandé, non obligatoire)

Ce n'est pas noté, mais fortement conseillé :

- Branche **main** : version stable
- Branches **feature/xxx** : nouvelles fonctionnalités
- Utiliser des Pull Requests pour valider les changements en équipe

6.4 Ce que le dépôt DOIT contenir à la remise

- Tout le code source (dossier **src/**)
- Le fichier **pom.xml**
- Les données (**jeux.csv**, **films.csv**, etc.) dans **src/main/resources/data/**
- Un **README.md** complet (voir modèle fourni)
- Le **.gitignore**
- Aucun dossier **target/**, **.idea/**, **out/**

7. Remise sur Teams

Le représentant de chaque équipe publie sur le canal du groupe, avant le **vendredi 11 septembre 2026 à 23h59** :

1. Un fichier **README.md** rempli à partir du modèle fourni (**README_Template.md**)
2. Ce **README** contient :
 - Le nom des membres de l'équipe
 - Le sujet choisi
 - Le **lien vers le dépôt GitHub PUBLIC**
 - Une brève description des fonctionnalités implémentées
 - Les instructions pour lancer le projet

Aucun code n'est déposé sur Teams. Uniquement le fichier **README** avec le lien **GitHub**. Vérifiez que votre lien fonctionne dans un navigateur privé (sinon, le prof ne peut pas y accéder).

Politique de retard

- Retard de 1 jour : pénalité de 10%
- Retard de 2 jours : pénalité de 25%
- Retard de 3 jours et plus : note de 0

8. Barème de correction (100 points)

primary !20 Critère	Points
Architecture MVC + structure des packages + interfaces	25
Modèle POO propre (encapsulation, immutabilité, séparation)	10
Tris à la main (au moins 3, avec Comparator)	15
Pagination fonctionnelle	10
Filtrage multi-critères qui se combinent	10
Recherche par texte en temps réel	5
Benchmark comparatif des tris	5
Interface JavaFX soignée (FXML + CSS)	10
Utilisation professionnelle de Git (commits réguliers, messages clairs, contribution répartie)	5
README complet, projet compile et lance sans erreur	5
primary !30 TOTAL	100

Points bonus (jusqu'à +10%) : selon les bonus spécifiques au sujet (voir PDF du sujet).

9. Ce qui vient au Laboratoire 3

Pour vous motiver à faire du CODE PROPRE, voici ce que vous ajouterez au même projet dans 3 semaines :

- Migration des données CSV vers **PostgreSQL** (pattern DAO)
- Système d'**authentification** (login/register, mots de passe hashés)
- Wishlist/favoris **persistants par utilisateur**
- **Multithreading** JavaFX (chargement asynchrone, barre de progression)
- Écran de **statistiques** avec charts (LineChart, PieChart, BarChart)
- Bonus : intégration d'une **API externe** (TMDb, Spotify, NHL, etc.)

Si vous respectez l'architecture demandée aujourd'hui, le Lab 3 sera un plaisir. Sinon, il sera un enfer.

10. Conseils pratiques

10.1 Répartition des rôles

Équipe de 2 :

- Personne A : Modèle + Service + Tris (backend)
- Personne B : FXML + Controller + CSS (frontend)

Équipe de 3 :

- Personne A : Modèle + Service (chargement CSV, filtrage, pagination)
- Personne B : Algorithmes de tri + benchmark
- Personne C : UI (FXML + Controller + CSS)

10.2 Étapes suggérées

1. Créer le dépôt GitHub, cloner en local, invitations
2. Créer le `pom.xml` et vérifier que JavaFX se lance ("Hello World")
3. Définir la classe du domaine (ex : `Film`) et son enum de genres
4. Écrire `LecteurCSV` qui retourne une liste de films
5. Créer le squelette UI : `TableView` vide + charger les données au démarrage
6. Ajouter le premier tri (Bubble Sort), l'appeler depuis un bouton
7. Ajouter les deux autres tris
8. Implémenter la pagination
9. Implémenter les filtres
10. Implémenter la recherche
11. Implémenter le comparateur/benchmark
12. Polish CSS, écrire le README, valider la remise

10.3 Erreurs de débutants à éviter

- Coder toute une fonctionnalité avant de la tester — codez petit, testez souvent
- Négliger le `module-info.java` — toujours `opens ... to javafx.fxml`
- Mettre les données brutes dans le code Java — toujours dans un CSV
- Faire un seul commit géant à la fin — commiter au fur et à mesure
- Attendre le dernier jour — 2 semaines et demi passent vite

11. Questions fréquentes

On peut utiliser ChatGPT / Copilot / Claude ?

Pour **comprendre** un concept : oui, comme un tuteur. Pour **générer du code entier** : non. Le prof reconnaîtra le style généré et pénalisera. L'objectif est que vous appreniez à coder.

Utiliser un outil pour vous aider à **déboguer** ou **comprendre un message d'erreur** est acceptable.

Notre application peut-elle inclure une base de données ?

Non. Le Lab 2 utilise uniquement des fichiers CSV. La base de données est le sujet du Lab 3.

Peut-on faire une autre langue d'interface (anglais) ?

Oui, l'interface peut être en français, en anglais ou bilingue. Le code (variables, méthodes) doit être **cohérent** dans une seule langue.

On peut ajouter des fonctionnalités non demandées ?

Oui, mais seulement APRÈS avoir terminé les fonctionnalités obligatoires. Comptez comme bonus.

Qui est responsable en cas de conflit d'équipe ?

En cas de conflit non résolu, contactez le prof **avant la dernière semaine** pour trouver une solution. Un membre qui ne contribue pas se verra attribuer une note individuelle inférieure.

Bon développement !

Rappel : ce projet est la base du Lab 3. Faites-le proprement.